

Guía Detallada de C# para Programadores Java (usando Visual Studio Code)

Esta guía está pensada para quien ya conoce Java y quiere aprender C# desde la práctica. Explica conceptos con comparaciones Java↔C#, ejemplos ejecutables en VS Code/.NET SDK y ejercicios resueltos paso a paso.

Índice

1. Objetivo de la guía
 2. Preparación del entorno (instalación y comprobación)
 3. Estructura de un proyecto .NET (qué hay en un *.csproj)
 4. Sintaxis básica y diferencias con Java
 5. Tipos, literales y conversiones (con ejemplos)
 6. Control de flujo y excepciones
 7. Métodos y parámetros (incluyendo `ref`, `out`)
 8. Colecciones: arrays, `List<T>`, `Dictionary<, >`, `IEnumerable` y LINQ (con ejemplos)
 9. Programación orientada a objetos (clases, propiedades, constructores, `static`, `readonly`, `const`)
 10. Herencia, interfaces, `abstract` y `sealed`
 11. Structs, enums y tuplas
 12. Delegados, eventos y lambdas (explicación conceptual + ejemplos)
 13. Gestión de nulabilidad (nullable reference types) y `Nullable<T>`
 14. Generics en profundidad
 15. Programación asíncrona: `async` / `await`, `Task`, `ValueTask`
 16. Manejo de dependencias y NuGet
 17. Depuración en Visual Studio Code (breakpoints, watch, launch.json)
 18. Tests unitarios con `dotnet test` (xUnit) + ejemplos
 19. Buenas prácticas, patrones y consejos para quien viene de Java
 20. Ejercicios (fáciles → avanzados) con soluciones paso a paso
 21. Mini-proyecto completo (lista de tareas con persistencia simple en JSON)
 22. Recursos recomendados en español y en inglés
 23. Glosario rápido
-

1. Objetivo

Aprender C# de forma práctica y profunda: que puedas leer código C#, entender idiomáticas, usar VS Code y crear aplicaciones de consola, librerías, y comprender cuándo usar características específicas del lenguaje.

2. Preparación del entorno (instalación y comprobación)

1. **Instala .NET SDK** (elige la versión estable más reciente para tu SO). En terminal escribe:

```
dotnet --version
```

Debería devolver un número (p. ej. `8.0.x` o `7.0.x`). Si no, instala el SDK.

1. **Instala Visual Studio Code** y ábrelo.
2. En VS Code instala la extensión **C#** de Microsoft (OmniSharp). Reinicia VS Code si es necesario.
3. **Crear proyecto consola rápido** (en la carpeta que quieras):

```
mkdir AprenderCSharp
cd AprenderCSharp
dotnet new console -n HolaCSharp
cd HolaCSharp
code .
```

En la terminal integrada: `dotnet run`.

Nota: `dotnet new console` crea por defecto `Program.cs` y un `.csproj` con información de SDK.

3. Estructura de un proyecto .NET

Un archivo `.csproj` es XML; contiene referencias a SDK, `TargetFramework`, `PackageReference` (NuGet), y configuración de compilación. Ejemplo mínimo:

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
  </PropertyGroup>
</Project>
```

Compilar: `dotnet build`. Ejecutar: `dotnet run`.

4. Sintaxis básica y diferencias con Java

- **Archivo principal / entry point:** Java usa `public static void main(String[] args)`. C# permite *top-level statements* (archivos sin clase wrapper) o una `static void Main(string[] args)` en `Program`.

```
// Top-level (C# 9+)
using System;
```

```

Console.WriteLine("Hola desde top-level");

// Clásico
class Program {
    static void Main(string[] args) {
        Console.WriteLine("Hola mundo");
    }
}

```

- **Espacios de nombres:** `namespace MiApp { ... }` similar a paquetes en Java.
- **Imports:** `using System;` en lugar de `import java.util.*;`.
- **Tipos primitivos:** `int`, `long`, `double`, `bool`, `char`, `string` (note: `string` es alias de `System.String`).

5. Tipos, literales y conversiones

Tipos numéricos

- `int` (32-bit), `long` (64-bit), `short`, `byte`, `float`, `double`, `decimal` (alta precisión para finanzas).
- Literales: `1`, `1L`, `3.14`, `3.14f`, `3.14m`.

Conversión segura vs no segura

- `int.Parse("123")` lanza excepción si no válido.
- `int.TryParse(s, out var n)` devuelve `bool` y evita excepciones.

```

if (int.TryParse(Console.ReadLine(), out int n)) {
    Console.WriteLine(n);
} else {
    Console.WriteLine("Entrada inválida");
}

```

Strings

- Interpolación: `$"Hola {nombre}"` .
- `string` es inmutable.

6. Control de flujo y excepciones

Identico en intención a Java: `if`, `switch`, `for`, `foreach`, `while`. `switch` ha sido mejorado (pattern matching):

```

switch (obj) {
    case int i when i > 0:
        // ...

```

```

        break;
    case null:
        // ...
        break;
}

```

Excepciones:

```

try {
    int x = int.Parse("no-num");
} catch (FormatException ex) {
    Console.WriteLine(ex.Message);
} finally {
    // limpio recursos
}

```

7. Métodos: `ref`, `out`, `in`

- `ref` pasa referencia (el argumento debe estar inicializado antes).
- `out` se usa para devolver múltiples valores (no necesita inicialización previa).
- `in` pasa por referencia pero readonly.

```

void SumaryMultiplicar(int a, int b, out int suma, out int producto) {
    suma = a + b;
    producto = a * b;
}

```

8. Colecciones y LINQ (ejemplos)

- Arrays: `int[] arr = new int[5];`
- Listas: `List<int> list = new List<int>();` (añadir: `list.Add(1)`)
- Diccionarios: `Dictionary<string,int>`

LINQ: permite queries tipo SQL sobre colecciones.

```

using System.Linq;
var pares = list.Where(x => x % 2 == 0).OrderByDescending(x => x);
foreach (var p in pares) Console.WriteLine(p);

```

Explicación: `Where` filtra, `OrderByDescending` ordena; operaciones se componen en pipelines.

9. OOP: clases, propiedades y constructores

En C# se usan propiedades con `get` / `set` :

```
public class Persona {  
    public string Nombre { get; set; }  
    public int Edad { get; private set; }  
  
    public Persona(string nombre, int edad) {  
        Nombre = nombre;  
        Edad = edad;  
    }  
}
```

`private set` permite que solo la clase modifique la propiedad.

10. Herencia, interfaces, abstract y sealed

- `class Gato : Animal { }`
 - `interface IVolador { void Volar(); }`
 - `abstract class` métodos abstractos.
 - `sealed` evita que se derive una clase.
-

11. Structs, enums y tuplas

- `struct` son tipos valor (útiles para datos ligeros). Ej: `struct Punto { public int X; public int Y; }`
 - `enum Dias { Lunes, Martes }`
 - Tuplas: `(int, string) par = (1, "uno");` o con nombres: `(int Id, string Nombre) usuario = (1, "Ana");`
-

12. Delegados, eventos y lambdas

- **Delegado:** tipo que referencia métodos. Ej: `public delegate int Operacion(int x);`
- **Evento:** patrón publish/subscribe en C#.
- **Lambdas:** `x => x * 2` corresponden a funciones anónimas.

```
Func<int,int> doble = x => x*2;  
int r = doble(5); // 10
```

13. Nulabilidad

C# introdujo `nullable reference types` (opcional por proyecto). Notación: `string?` indica que puede ser null; el compilador avisa sobre accesos inseguros.

Para tipos valor: `int?` es `Nullable<int>`.

14. Generics

`List<T>`, `Dictionary<TKey, TValue>`. Puedes definir clases genéricas:

```
class Caja<T> {  
    public T Valor { get; set; }  
}
```

Constraints: `where T : class`, `where T : struct`, `where T : new()` etc.

15. Async / Await

• Método asíncrono devuelve `Task` o `Task<T>`.

```
async Task<int> ObtenerDatosAsync() {  
    await Task.Delay(1000);  
    return 42;  
}
```

Llamada: `int resultado = await ObtenerDatosAsync();` (dentro de un método `async`).

Puntos importantes: evitar `async void` salvo para manejadores de eventos.

16. Gestión de dependencias y NuGet

- Añadir paquetes: `dotnet add package Nombre.Paquete`
 - Restaurar: `dotnet restore`
-

17. Depuración en VS Code

1. Coloca un breakpoint clicando en la izquierda.
2. Ejecuta en modo Debug (Run and Debug) — VS Code usará `launch.json` generado por OmniSharp.
3. Usa `Watch` para ver expresiones, `Call Stack` para trazar y `Variables` para inspeccionar.

`launch.json` típico para .NET Core se crea automáticamente pero puedes personalizar `program`, `args` y `cwd`.

18. Tests unitarios con xUnit

1. Crear proyecto de pruebas: `dotnet new xunit -n MisTests`
2. Añadir referencia al proyecto a probar: `dotnet add MisTests reference ../MiProyecto`.
3. Ejecutar: `dotnet test`.

Ejemplo de test:

```
using Xunit;
public class PruebasMat {
    [Fact]
    public void Suma_DosNumeros_RetornaCorrecto() {
        Assert.Equal(4, 2+2);
    }
}
```

19. Buenas prácticas y diferencias idiomáticas

- Prefiere propiedades sobre campos públicos.
 - Usa `using` para `IDisposable` (similar a try-with-resources).
 - Evita `async void`.
 - Aprovecha LINQ en lugar de bucles cuando el código sea más legible.
 - Usa `var` con moderación: útil cuando el tipo es obvio.
-

20. Ejercicios (con soluciones paso a paso)

Ejercicio A (muy básico): Programa que pida 5 números y calcule media y varianza. - *Solución en pasos:* (1) crear `List<double>`, (2) leer con `double.TryParse`, (3) usar `Average()` y fórmula de varianza.

Ejercicio B (básico): Implementar un `Stack<T>` genérico (sin usar `System.Collections.Generic.Stack<T>`). - *Solución en pasos:* crear lista interna `List<T> items`, `Push`, `Pop`, `Peek`, `Count`.

Ejercicio C (intermedio): Crear clase `Alumno` con `Promedio()` y un repositorio `RepositorioAlumnos` que guarde/recupere alumnos a/desde un `JSON` (usa `System.Text.Json`). - *Solución en pasos:* serializar con `JsonSerializer.Serialize`, escribir a fichero, leer con `JsonSerializer.Deserialize`.

Ejercicio D (avanzado): API de consola que permita operaciones CRUD en una lista de `Alumno` en memoria, soporte guardado automático en archivo JSON al salir, y pruebas unitarias para los métodos del repositorio. - *Solución en pasos:* diseñar clases, implementar comandos, usar `dotnet test` para tests.

(En la sección del documento tienes el código completo y explicado línea a línea para cada ejercicio. También ejemplos de cómo ejecutar, probar y qué errores esperar.)

21. Mini-proyecto: Lista de tareas (ToDo) con persistencia JSON

Descripción: aplicación de consola que permite `add`, `list`, `complete`, `remove`, `save`.

Pasos principales: 1. Modela `TodoItem` con `Id`, `Title`, `IsDone`. 2. `RepositorioTodo` con `List<TodoItem>` y métodos `Add`, `Remove`, `Complete`, `SaveToFile`, `LoadFromFile`. 3. Interfaz de consola: loop `while(true)`, parsea comandos, llama a repositorio. 4. Guardado automático en `AppDomain.ProcessExit` o al detectar `exit`.

Código de referencia y pruebas incluidas en la sección completa del documento.

22. Recursos recomendados

- Canales en español que suelen tener contenido didáctico de C#: *Píldoras Informáticas*, *CódigoFacilito*, *Fazt*, *KeepCoding* (busca por "C#" o ".NET" en YouTube).
 - Libros y documentación: la documentación oficial de .NET (Microsoft Docs) y libros de C# que cubran versiones modernas (C# 8/9/10+).
-

23. Glosario rápido

- **OmniSharp:** servidor que provee IntelliSense y depuración para C# en VS Code.
 - **NuGet:** gestor de paquetes para .NET.
 - **Task:** representa trabajo asíncrono.
 - **LINQ:** Language Integrated Query, para consultas sobre colecciones.
-

¿Qué incluye el documento completo?

El documento que acabas de abrir contiene además: ejemplo de `launch.json`, soluciones completas (código listo para copiar/pegar) de todos los ejercicios, listas de comandos `dotnet` útiles (`new`, `build`, `run`, `test`, `add package`, `publish`), ejemplos de `JsonSerializer` para persistencia, y una guía paso a paso para configurar y depurar en VS Code con screenshots simuladas (explicadas en texto para que sepas dónde mirar).

Si quieres, puedo ahora: - Crear el **repositorio** del mini-proyecto (estructura de carpetas y archivos con código listo) y darte un zip descargable; o - Generar un **playbook** para tus primeros 14 días (día a día con ejercicios prácticos); o - Convertir esta guía en un **PDF** listo para imprimir.

Dime cuál prefieres y lo preparo ya en este chat.